

Entrega 3 DESARROLLADOR 1

- VISTAS

1. Vista 1:

/*

=====

Archivo: vw_ventas_detalladas.sql

Entrega: Entrega 3

Descripción:

Crea una vista que permite consultar el detalle de las ventas, incluyendo información del cliente, empleado y productos vendidos.

Regla de negocio:

Una venta pertenece a un cliente y a un empleado y puede contener uno o varios productos.

El subtotal se calcula mediante:

$\text{cantidad} * \text{precio_unitario}$

Dependencias:

- venta
- cliente
- empleado
- producto
- producto_venta

=====

*/

USE bd_tradicion;

DROP VIEW IF EXISTS vw_ventas_detalladas;

CREATE VIEW vw_ventas_detalladas AS

SELECT

```

v.id_venta,
v.fecha_venta,
v.asunto,
c.id_cliente,
c.nombre AS cliente,
e.id_empleado,
e.nombre AS empleado,
p.id_producto,
p.nombre AS producto,
pv.cantidad,
pv.precio_unitario,
(pv.cantidad * pv.precio_unitario) AS subtotal,
v.monto AS total_venta
FROM venta AS v
INNER JOIN cliente AS c
    ON v.id_cliente = c.id_cliente
INNER JOIN empleado AS e
    ON v.id_empleado = e.id_empleado
INNER JOIN producto_venta AS pv
    ON v.id_venta = pv.id_venta
INNER JOIN producto AS p
    ON pv.id_producto = p.id_producto;

```

2. Vista 2:

/*

=====

Archivo: vw_inventario_productos.sql

Entrega: Entrega 3

Descripción:

Vista destinada a consultar el inventario actual de productos
y determinar automáticamente su estado de disponibilidad.

Regla de negocio:

- 0 unidades: SIN STOCK
- Entre 1 y 10 unidades: STOCK BAJO
- Más de 10 unidades: DISPONIBLE

El estado del stock no se almacena físicamente, sino que se calcula mediante la vista para evitar información redundante.

Dependencias:

- producto

=====

*/

USE bd_tradicion;

DROP VIEW IF EXISTS vw_inventario_productos;

CREATE VIEW vw_inventario_productos AS

SELECT

id_producto,

nombre AS producto,

descripcion,

unidades AS stock_actual,

precio_unitario,

CASE

WHEN unidades = 0 THEN 'SIN STOCK'

```

        WHEN unidades <= 10 THEN 'STOCK BAJO'

        ELSE 'DISPONIBLE'

    END AS estado_stock

FROM producto;

```

- TRIGGERS

1. Trigger 1

* Cuando se agregue un producto a una venta, el sistema debe comprobar que exista suficiente inventario y, si existe, descontar automáticamente la cantidad vendida. *

/*

=====

Archivo: trg_validar_stock_venta.sql

Entrega: Entrega 3

Descripción:

Valida la disponibilidad de inventario antes de registrar un producto dentro de una venta.

Regla de negocio:

No se permite registrar una cantidad menor o igual a cero ni vender una cantidad superior al inventario disponible.

Dependencias:

- producto

- producto_venta

=====

*/

USE bd_tradicion;

DELIMITER \$\$

DROP TRIGGER IF EXISTS trg_validar_stock_venta\$\$

```
CREATE TRIGGER trg_validar_stock_venta
BEFORE INSERT ON producto_venta
FOR EACH ROW
BEGIN
    DECLARE stock_disponible INT;

    SELECT unidades
    INTO stock_disponible
    FROM producto
    WHERE id_producto = NEW.id_producto;

    IF stock_disponible IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El producto no existe';
    END IF;

    IF NEW.cantidad <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La cantidad vendida debe ser mayor que cero';
    END IF;

    IF stock_disponible < NEW.cantidad THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Stock insuficiente para realizar la venta';
    END IF;
END$$

DELIMITER ;
```

2. Trigger 2 Descontar stock después de una venta

/*

=====

Archivo: trg_descontar_stock_venta.sql

Entrega: Entrega 3

Descripción:

Actualiza automáticamente el inventario de productos después de registrar un producto dentro de una venta.

Regla de negocio:

Cada producto vendido debe descontarse del inventario según la cantidad registrada en producto_venta.

Este trigger trabaja junto con trg_validar_stock_venta, que comprueba previamente que exista suficiente inventario.

Dependencias:

- producto
- producto_venta

=====

*/

USE bd_tradicion;

DELIMITER \$\$

DROP TRIGGER IF EXISTS trg_descontar_stock_venta\$\$

CREATE TRIGGER trg_descontar_stock_venta

AFTER INSERT ON producto_venta

FOR EACH ROW

BEGIN

 UPDATE producto

 SET unidades = unidades - NEW.cantidad

 WHERE id_producto = NEW.id_producto;

END\$\$

DELIMITER ;

- **Prueba 1: Venta con Stock suficiente**

Tabla CLIENTES:

```

1 • USE bd_tradicion;
2
3
4 • SELECT * FROM cliente;
5 • SELECT * FROM empleado;
6 • SELECT * FROM producto;
7 • SELECT * FROM venta;

```

Autodisabled. manually current toggle

id_cliente	nombre	telefono	correo	direccion
1	David	1222 4432	aridavid@gmail.com	35AvC 10A-61 zona 8
4	andy	1116 6666	andy@gmail.com	Calle Huerta Marcos 15
5	Mario	2123 5443	margonzalez@gmail.com	NULL
NULL	NULL	NULL	NULL	NULL

Tabla EMPLEADOS:

```

1 • USE bd_tradicion;
2
3
4 • SELECT * FROM cliente;
5 • SELECT * FROM empleado;
6 • SELECT * FROM producto;
7 • SELECT * FROM venta;

```

id_empleado	nombre	dpi	telefono	salario	fecha_contratacion	estado
1	Mary	123...	1233 4...	4002.28	2024-11-12	Activo
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Tabla PRODUCTOS:

Query 1 x bd_tradicion_mysql*

```

1 • USE bd_tradicion;
2
3
4 • SELECT * FROM cliente;
5 • SELECT * FROM empleado;
6 • SELECT * FROM producto;
7 • SELECT * FROM venta;

```

Autodisabled. manually current toggle

id_producto	nombre	descripcion	unidades	precio_unitario
1	Nacho...	NULL	20	15.00
NULL	NULL	NULL	NULL	NULL

Tabla VENTAS:

Query 1: bd_tradicion_mysql

```
1 • USE bd_tradicion;  
2  
3  
4 • SELECT * FROM cliente;  
5 • SELECT * FROM empleado;  
6 • SELECT * FROM producto;  
7 • SELECT * FROM venta;
```

Automatic config disabled. Use the manual button to manually get the current caret position and toggle automatic.

id_venta	asunto	fecha_venta	monto	id_empleado	id_cliente
1	compr...	2026-09-15	75.00	1	1

cliente 3 empleado 4 producto 5 venta 6 x Apply Context Help Snippets

CANTIDAD SUFICIENTE

INSERT INTO producto_venta

(id_venta, id_producto, cantidad, precio_unitario)

VALUES

(1, 1, 5, 15.00);

SELECCIONAR TABLA

SELECT id_producto, nombre, unidades

FROM producto

WHERE id_producto = 1;

Query 8: bd_tradicion_mysql

```
1 • USE bd_tradicion;  
2  
3  
4 • SELECT id_producto, nombre, unidades  
5 FROM producto  
6 WHERE id_producto = 1;
```

id_producto	nombre	unidades
1	Nachos barbacoa	15

producto 8 x Apply Co

Vista vw_ventas_detalladas:

```
4 • SELECT *
5 FROM vw_ventas_detalladas;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	id_venta	fecha_venta	asunto	id_cliente	cliente	id_empleado	empleado	id_producto	producto	cantidad	precio_unitario	subtotal	total_venta
▶	1	2026-09-15	compra de nachos de barbacoa	1	David	1	Mary	1	Nachos barbacoa	5	15.00	75.00	75.00

• Prueba 2: Stock insuficiente

con stock = 15, vamos a intentar vender 16 unidades

segunda venta:

INSERT INTO venta

(id_venta, asunto, fecha_venta, monto, id_empleado, id_cliente)

VALUES

(2, 'Prueba stock insuficiente', '2026-09-15', 240.00, 1, 1);

INSERT INTO producto_venta

(id_venta, id_producto, cantidad, precio_unitario)

VALUES

(2, 1, 16, 15.00);

Debe aparecer un error al intentar insertar datos por stock insuficiente

29 19:08:44 INSERT INTO producto_venta (id_venta, id_producto, cantidad, precio_unitario) VALUES (2, 1, 16, 15.00) Error Code: 1644. Stock insuficiente para realizar la venta

Seleccionar la tabla de productos

SELECT id_producto, nombre, unidades

FROM producto

WHERE id_producto = 1;

Continúa con la misma cantidad sin aplicar cambios al stock

```
4 • SELECT id_producto, nombre, unidades
5 FROM producto
6 WHERE id_producto = 1;
```

current toggle

id_producto	nombre	unidades
1	Nachos barbecoa	15
*	NULL	NULL

producto 10 x Apply Context Help

Output

Seleccionar la tabla que relaciona los productos y ventas para comprobar que no se inserto nada

SELECT *

FROM producto_venta

WHERE id_venta = 2;

```
4 • SELECT *
5 FROM producto_venta
6 WHERE id_venta = 2;
```

CL

id_venta	id_producto	cantidad	precio_unitario
*	NULL	NULL	NULL

producto_venta 11 x Apply Conl

- PROCEDIMIENTOS

1. Procedimiento 1: Centralizar el registro de una venta y su producto

Como una venta puede contener muchos productos, en un sistema definitivo normalmente no convendría diseñar un procedimiento que solo admita un producto. Para esta entrega podemos hacer un procedimiento enfocado en agregar una línea de venta completa, y después la aplicación puede llamarlo por cada producto o evolucionarlo a JSON/transacción múltiple.

```
USE bd_tradicion;
```

```
DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS sp_registrar_venta$$
```

```
CREATE PROCEDURE sp_registrar_venta(
```

```
    IN p_asunto VARCHAR(200),
```

```
    IN p_fecha_venta DATE,
```

```
    IN p_id_empleado INT,
```

```
    IN p_id_cliente INT,
```

```
    IN p_id_producto INT,
```

```
    IN p_cantidad INT,
```

```
    IN p_precio_unitario DECIMAL(10,2)
```

```
)
```

```
BEGIN
```

```
    DECLARE v_id_venta INT;
```

```
DECLARE v_total DECIMAL(10,2);
```

```
/*
```

```
Si ocurre cualquier error durante la operación,  
se revierten todos los cambios.
```

```
*/
```

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    RESIGNAL;
```

```
END;
```

```
START TRANSACTION;
```

```
/* Validar cliente */
```

```
IF NOT EXISTS (
```

```
    SELECT 1
```

```
    FROM cliente
```

```
    WHERE id_cliente = p_id_cliente
```

```
) THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'El cliente indicado no existe';
```

```
END IF;
```

```
/* Validar empleado */
```

```
IF NOT EXISTS (
```

```

SELECT 1
FROM empleado
WHERE id_empleado = p_id_empleado
) THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'El empleado indicado no existe';
END IF;

/* Validar cantidad */
IF p_cantidad <= 0 THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'La cantidad debe ser mayor que cero';
END IF;

SET v_total = p_cantidad * p_precio_unitario;

/*
    Generamos el siguiente ID porque venta actualmente
    no utiliza AUTO_INCREMENT.
*/
SELECT COALESCE(MAX(id_venta), 0) + 1
INTO v_id_venta
FROM venta;

INSERT INTO venta (
    id_venta,

```

```
    asunto,  
    fecha_venta,  
    monto,  
    id_empleado,  
    id_cliente  
)  
VALUES (  
    v_id_venta,  
    p_asunto,  
    p_fecha_venta,  
    v_total,  
    p_id_empleado,  
    p_id_cliente  
);
```

```
/*
```

Al ejecutar este INSERT entran en acción:

- trg_validar_stock_venta
- trg_descontar_stock_venta

```
*/
```

```
INSERT INTO producto_venta (  
    id_venta,  
    id_producto,  
    cantidad,  
    precio_unitario  
)
```

```
VALUES (
    v_id_venta,
    p_id_producto,
    p_cantidad,
    p_precio_unitario
);
```

```
COMMIT;
```

```
/* Devolver información de la venta creada */
```

```
SELECT
    v_id_venta AS id_venta,
    v_total AS total,
    'Venta registrada correctamente' AS mensaje;
```

```
END$$
```

```
DELIMITER ;
```

Se comprueba que existe:

```
SHOW PROCEDURE STATUS
```

```
WHERE Db = 'bd_tradicion';
```

	Db	Name	Type	Language	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection
►	bd_tradicion	sp_registrar_venta	PROCEDURE	SQL	root@localhost	2026-09-15 19:28:24	2026-09-15 19:28:24	DEFINER		utf8mb4	utf8mb4_0900_ai_ci

Prueba:

Se cuenta con:

Producto 1 = Nachos barbacoa

Stock = 15

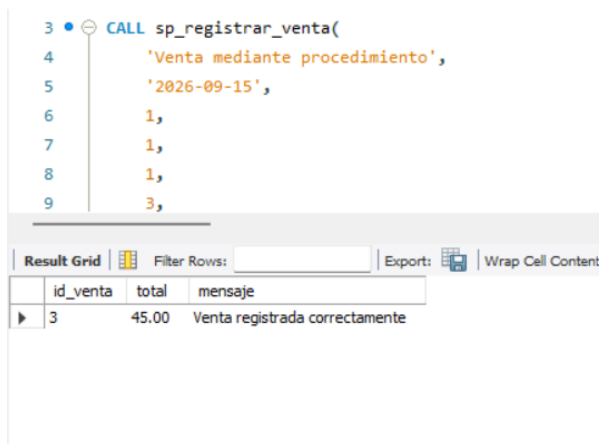
Precio = Q15

Cliente 1 = David

Empleado 1 = Mary

Entonces se puede probar y crear la venta #3:

```
CALL sp_registrar_venta(  
    'Venta mediante procedimiento',  
    '2026-09-15',  
    1,  
    1,  
    1,  
    3,  
    15.00  
);
```



The screenshot shows a SQL IDE with a query editor and a result grid. The query editor contains the following SQL code:

```
3 CALL sp_registrar_venta(  
4     'Venta mediante procedimiento',  
5     '2026-09-15',  
6     1,  
7     1,  
8     1,  
9     3,  
10    15.00  
11 );
```

The result grid is displayed below the query editor. It has a header row with columns: id_venta, total, and mensaje. The first row of data shows id_venta: 3, total: 45.00, and mensaje: Venta registrada correctamente.

id_venta	total	mensaje
3	45.00	Venta registrada correctamente

Seleccionar la vista de ventas detalladas con el registro más reciente:

```
SELECT *  
  
FROM vw_ventas_detalladas  
  
ORDER BY id_venta;
```

```
3 • SELECT *  
4 FROM vw_ventas_detalladas  
5 ORDER BY id_venta;
```

Result Grid Filter Rows: Export: Wrap Cell Content:													
	id_venta	fecha_venta	asunto	id_cliente	cliente	id_empleado	empleado	id_producto	producto	cantidad	precio_unitario	subtotal	total_venta
▶	1	2026-09-15	compra de nachos de barbacoa	1	David	1	Mary	1	Nachos barbacoa	5	15.00	75.00	75.00
	3	2026-09-15	Venta mediante procedimiento	1	David	1	Mary	1	Nachos barbacoa	3	15.00	45.00	45.00

Seleccionar la vista del inventario de productos y comprobar que se actualizo:

```
SELECT *  
  
FROM vw_inventario_productos  
  
WHERE id_producto = 1;
```

```
3 • SELECT *  
4 FROM vw_inventario_productos  
5 WHERE id_producto = 1;
```

Result Grid Filter Rows: Export: Wrap Cell Content:						
	id_producto	producto	descripcion	stock_actual	precio_unitario	estado_stock
▶	1	Nachos barbacoa	NULL	12	15.00	DISPONIBLE

- El stock paso de 15 a 12 y aparece la nueva venta, por lo tanto el procedimiento 1 queda terminado

2. Procedimiento 2: Centralizar el registro de producción

Se quiere que el procedimiento 2 haga

Registrar producción



Comprobar producto



Comprobar receta



Comprobar materias primas



Registrar PRODUCCION



Registrar DETALLE_FABRICACION



Descontar materias primas



Aumentar stock del producto



COMMIT

Crear sp_registrar_produccion

USE bd_tradicion;

DELIMITER \$\$

```
DROP PROCEDURE IF EXISTS sp_registrar_produccion$$
```

```
CREATE PROCEDURE sp_registrar_produccion(
```

```
    IN p_nombre VARCHAR(200),
```

```
    IN p_descripcion VARCHAR(200),
```

```
    IN p_fecha_produccion DATE,
```

```
    IN p_id_producto INT,
```

```
    IN p_cantidad_producida INT
```

```
)
```

```
BEGIN
```

```
    DECLARE v_id_produccion INT;
```

```
    DECLARE v_materias_receta INT DEFAULT 0;
```

```
    DECLARE v_materias_insuficientes INT DEFAULT 0;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
    BEGIN
```

```
        ROLLBACK;
```

```
        RESIGNAL;
```

```
    END;
```

```
    START TRANSACTION;
```

```
    IF p_cantidad_producida <= 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'La cantidad producida debe ser mayor que cero';
```

END IF;

IF NOT EXISTS (

 SELECT 1

 FROM producto

 WHERE id_producto = p_id_producto

) THEN

 SIGNAL SQLSTATE '45000'

 SET MESSAGE_TEXT = 'El producto indicado no existe';

END IF;

SELECT COUNT(*)

INTO v_materias_receta

FROM materia_prima_producto

WHERE id_producto = p_id_producto;

IF v_materias_receta = 0 THEN

 SIGNAL SQLSTATE '45000'

 SET MESSAGE_TEXT = 'El producto no tiene materias primas registradas';

END IF;

SELECT COUNT(*)

INTO v_materias_insuficientes

FROM materia_prima_producto AS mpp

INNER JOIN materia_prima AS mp

 ON mp.id_materia_prima = mpp.id_materia_prima

```
WHERE mpp.id_producto = p_id_producto  
AND mp.cantidad < (mpp.cantidad * p_cantidad_producida);
```

```
IF v_materias_insuficientes > 0 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT =  
        'Materia prima insuficiente para realizar la produccion';  
END IF;
```

```
SELECT COALESCE(MAX(id_produccion), 0) + 1  
INTO v_id_produccion  
FROM produccion;
```

```
INSERT INTO produccion (  
    id_produccion,  
    nombre,  
    descripcion,  
    fecha_produccion  
)
```

```
VALUES (  
    v_id_produccion,  
    p_nombre,  
    p_descripcion,  
    p_fecha_produccion  
);
```

```
INSERT INTO detalle_fabricacion (
```

```
    id_produccion,
```

```
    id_producto,
```

```
    cantidad_producida
```

```
)
```

```
VALUES (
```

```
    v_id_produccion,
```

```
    p_id_producto,
```

```
    p_cantidad_producida
```

```
);
```

```
/* Descontar materia prima */
```

```
UPDATE materia_prima AS mp
```

```
SET mp.cantidad = mp.cantidad - (
```

```
    SELECT mpp.cantidad * p_cantidad_producida
```

```
    FROM materia_prima_producto AS mpp
```

```
    WHERE mpp.id_materia_prima = mp.id_materia_prima
```

```
    AND mpp.id_producto = p_id_producto
```

```
)
```

```
WHERE mp.id_materia_prima IN (
```

```
    SELECT mpp2.id_materia_prima
```

```
    FROM materia_prima_producto AS mpp2
```

```
    WHERE mpp2.id_producto = p_id_producto
```

```
);
```

```
/* Aumentar inventario del producto terminado */
```

UPDATE producto

SET unidades = unidades + p_cantidad_producida

WHERE id_producto = p_id_producto;

COMMIT;

SELECT

v_id_produccion AS id_produccion,

p_id_producto AS id_producto,

p_cantidad_producida AS cantidad_producida,

'Produccion registrada correctamente' AS mensaje;

END\$\$

DELIMITER ;

Se comprueba que fue creado:

```
1 • USE bd_tradicion;
2
3 • SHOW PROCEDURE STATUS
4   WHERE Db = 'bd_tradicion';
```

	Db	Name	Type	Language	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection
▶	bd_tradicion	sp_registrar_produccion	PROCEDURE	SQL	root@localhost	2026-09-15 19:58:31	2026-09-15 19:58:31	DEFINER		utf8mb4	utf8mb4_0900_...
	bd_tradicion	sp_registrar_venta	PROCEDURE	SQL	root@localhost	2026-09-15 19:28:24	2026-09-15 19:28:24	DEFINER		utf8mb4	utf8mb4_0900_...

Realizar producción de 10 Nachos

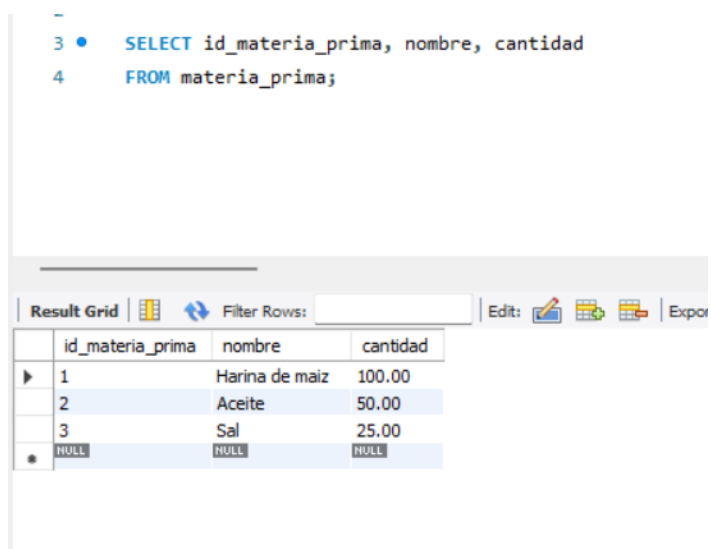
- Se sabe que se tienen 12 nachos en Stock
- Para fabricar 10 unidades, según nuestros datos de prueba se necesitarán

Harina: $10 \times 0.50 = 5.00$

Aceite: $10 \times 0.10 = 1.00$

Sal: $10 \times 0.05 = 0.50$

```
3 • SELECT id_materia_prima, nombre, cantidad
4 FROM materia_prima;
```



The screenshot shows a database interface with a SQL query editor at the top and a 'Result Grid' below it. The query is: `SELECT id_materia_prima, nombre, cantidad FROM materia_prima;`. The 'Result Grid' displays a table with three columns: `id_materia_prima`, `nombre`, and `cantidad`. The table contains three rows of data: (1, 'Harina de maiz', 100.00), (2, 'Aceite', 50.00), and (3, 'Sal', 25.00). A fourth row is highlighted with a blue background and contains NULL values for all three columns. The interface also includes a 'Filter Rows' field and buttons for 'Edit', 'Export', and 'Import'.

	id_materia_prima	nombre	cantidad
▶	1	Harina de maiz	100.00
	2	Aceite	50.00
	3	Sal	25.00
*	NULL	NULL	NULL

```
CALL sp_registrar_produccion(  
    'Produccion de Nachos',  
    'Produccion de prueba para Nachos barbacoa',  
    '2026-09-15',  
    1,  
    10  
);
```

```

1 • USE bd_tradicion;
2
3 • CALL sp_registrar_produccion(
4     'Produccion de Nachos',
5     'Produccion de prueba para Nachos barbacoa',
6     '2026-09-15',
7     1,
8     10
9 );

```

id_produccion	id_producto	cantidad_producida	mensaje
1	1	10	Produccion registrada correctamente

Realizar SELECT en las tablas para comprobar que la información fue actualizada correctamente:

Producción:

```

6 • SELECT *
7   FROM detalle_fabricacion;
8
9 • SELECT id_materia_prima, nombre, cantidad
10  FROM materia_prima;
11
12 • SELECT *
13  FROM vw_inventario_productos
14  WHERE id_producto = 1;

```

id_produccion	nombre	descripcion	fecha_produccion
1	Produccion de Nachos	Produccion de prueba para Nachos barbacoa	2026-09-15

produccion 33 x detalle_fabricacion 34 materia_prima 35 vw_inventario_productos 36

Detalle de fabricación:

```

6 • SELECT *
7   FROM detalle_fabricacion;
8
9 • SELECT id_materia_prima, nombre, cantidad
10  FROM materia_prima;
11
12 • SELECT *
13  FROM vw_inventario_productos
14  WHERE id_producto = 1;

```

id_produccion	id_producto	cantidad_producida
1	1	10

produccion 33 detalle_fabricacion 34 materia_prima 35 vw_inventario_productos 36

Materia prima:

Query 1: bd_tradicion_mysql

```
6 • SELECT *
7   FROM detalle_fabricacion;
8
9 • SELECT id_materia_prima, nombre, cantidad
10  FROM materia_prima;
11
12 • SELECT *
13  FROM vw_inventario_productos
14  WHERE id_producto = 1;
```

Result Grid

	id_materia_prima	nombre	cantidad
1	1	Harina de maiz	95.00
2	2	Aceite	49.00
3	3	Sal	24.50
*	NULL	NULL	NULL

produccion 33 detalle_fabricacion 34 materia_prima 35 vw_inventario_productos 36

Producto:

Query 1: bd_tradicion_mysql

```
6 • SELECT *
7   FROM detalle_fabricacion;
8
9 • SELECT id_materia_prima, nombre, cantidad
10  FROM materia_prima;
11
12 • SELECT *
13  FROM vw_inventario_productos
14  WHERE id_producto = 1;
```

Result Grid

	id_producto	producto	descripcion	stock_actual	precio_unitario	estado_stock
1	1	Nachos barbacoa	NULL	22	15.00	DISPONIBLE

produccion 33 detalle_fabricacion 34 materia_prima 35 vw_inventario_productos 36